

S26CS6.401 Software Engineering

Project 3 Report

HealthSync

Team 2

Team Member	Roll Number
Kartik Vij	2022102060
Kushagra Dhingra	2022115004
Lakshmi Dedeepya	2024801007
Prithvi Karthik	2022101020
Samagra Bharti	2022115007

Repository link: <https://github.com/Prithvi-k/SE-Project-3>

Date: April 25, 2026

Contents

1	Overview	1
2	Task 1: Requirements and Subsystems	1
2.1	Functional requirements	1
2.2	Non-functional requirements	2
2.3	Key architecturally significant requirements (ASRs)	4
2.4	Subsystem overview	5
3	Task 2: Architecture Framework	7
3.1	Stakeholders	7
3.2	Architecture Views	7
3.2.1	Architectural Overview	7
3.2.2	API Design View	8
3.2.3	Frontend and Backend Structure	8
3.2.4	Frontend C3 Diagram	9
3.2.5	Backend C3 Diagram	9
3.2.6	Interaction Flow	10
3.2.7	Meal Logging Sequence	11
3.2.8	Workout Logging Sequence	11
3.3	Major Design Decisions (Architecture Decision Records)	12
3.3.1	ADR-0001: Strict layered monolith architecture	12
3.3.2	ADR-0002: Authenticated backend API boundary for all user data writes	12
3.3.3	ADR-0003: Denormalised Firestore meal document	13
3.3.4	ADR-0004: Strategy + Chain-of-Responsibility for nutrition lookup	13
3.3.5	ADR-0005: Graceful dashboard degradation under partial service failures	14
4	Task 3: Architectural Tactics and Patterns	15
4.1	Architectural tactics	15
4.2	Implementation patterns	16
4.2.1	Pattern 1: Strategy Pattern (Recommendation Engine)	16
4.2.2	Pattern 2: Repository Pattern	17
5	Task 4: Prototype Implementation and Architecture Analysis	19
5.1	Implemented prototype	19
5.2	Architecture comparison: layered monolith vs. microservices	23
5.3	NFR quantification	23
6	Reflection and Lessons Learned	25

1 Overview

HealthSync is an integrated health tracking platform that combines meal logging, workout tracking, health goal management, progress analytics, and personalized workout recommendations in a single system. The motivation is to reduce the fragmentation users face when they rely on multiple apps to track diet, exercise, and fitness goals.

2 Task 1: Requirements and Subsystems

2.1 Functional requirements

The initial proposal defined the core functional scope: account management, meal logging, workout tracking, goals, dashboard, analytics, and recommendations. During implementation, several flows were expanded and clarified to make the system usable end-to-end.

ID	Requirement	Success criteria
FR-1	Users shall be able to register, login/logout, and maintain an authenticated session	POST <code>/auth/register</code> returns 201; POST <code>/auth/login</code> returns a valid Firebase ID token; POST <code>/auth/logout</code> clears the client token; any <code>/api/</code> call without a valid token returns HTTP 401.
FR-2	Users shall be able to sign in using email/password and Google OAuth	Both providers issue a Firebase ID token accepted by <code>get_current_user</code> ; neither path exposes any credential in the frontend bundle or URL.
FR-3	Authenticated users shall be able to create and update a personal profile	GET <code>/api/v1/profile</code> returns all stored fields; PUT <code>/api/v1/profile</code> with changed fields returns 200 and the updated document; subsequent GET reflects the change.
FR-4	Authenticated users shall be able to log a meal with meal type, date, time, and one or more food items	POST <code>/api/v1/meals</code> with a valid payload returns 201; the meal is retrievable via GET <code>/api/v1/meals?dateFrom=...</code> ; a request with zero items or a missing required field returns 422.
FR-5	Each meal item shall include name, quantity, unit, calories, protein, carbs, and fat	All seven fields present and non-null; quantity > 0; calories, protein, carbs, fat ≥ 0; Pydantic validation rejects out-of-range values with 422.
FR-6	The system shall calculate and persist meal totals server-side	Totals in the stored meal document equal the server-computed sum across items; submitting a payload with manually inflated client totals has no effect on the persisted values.

ID	Requirement	Success criteria
FR-7	Users shall be able to upload a meal photo for automatic food detection and nutrition autofill	POST /api/v1/meals/analyze-image with a valid image returns at least one detected food item with non-zero macro estimates; an unrecognisable image returns a structured error (no_food_detected), not a 500.
FR-8	Authenticated users shall be able to log workouts	POST /api/v1/workouts returns 201; type, duration (> 0), intensity, calories burned (≥ 0) are stored; missing required fields return 422.
FR-9	Users shall be able to set and update daily goals	PUT /api/v1/goals creates or replaces the active goal; only one active goal exists per user at any time; GET /api/v1/goals returns it.
FR-10	Users shall be able to view a unified daily dashboard	GET /api/v1/dashboard?date=... returns calories_in, calories_out, net_calories, macro breakdown, meal count, and workout count; response includes all fields even when no data exists for that day (zeroed values, not missing keys).
FR-11	Users shall be able to view history of logged meals and workouts and delete entries	History endpoint returns items for the requested date range; DELETE /api/v1/meals/{id} returns 204 and the entry is absent from subsequent history calls; deleting a non-existent ID returns 404.
FR-12	Users shall be able to view progress analytics across a date range	GET /api/v1/analytics?dateFrom=...&dateTo=... returns trend data for every day in the range; requesting a future range returns an empty dataset, not an error.
FR-13	The system shall provide personalized workout recommendations based on goals and historical behavior	GET /api/v1/recommendations returns ≥ 1 recommendation when the user has an active goal; the recommendation strategy matches the goal type (strength / cardio / weight-loss).

2.2 Non-functional requirements

The proposal identified six quality attributes: scalability, performance, extensibility, maintainability, security/privacy, and reliability/testability. We split Reliability and Testability into separate rows and add Data Integrity and Observability based on implementation evidence.

ID	Attribute	Necessity	Success threshold
NFR-1	Security / Privacy	All <code>/api/</code> endpoints reject unauthenticated requests; reads/writes scoped to the authenticated UID only.	0 endpoints accessible without a valid Firebase ID token; <code>gitleaks</code> : 0 secrets in HEAD; <code>slowapi</code> rate limits active on auth and image-analysis routes.
NFR-2	Data Integrity	Business-logic aggregates computed server-side; input validated at API boundary.	0 client-trusted aggregate values in any <code>POST</code> schema; $\geq 87\%$ of input fields carry explicit Pydantic range / format constraints.
NFR-3	Performance	Firestore reads sub-linear in history size; initial JS bundle small for fast initial load.	≤ 1 Firestore read per meal-list page (<code>N+1</code> fix); initial JS bundle ≤ 150 KB gzip (measured: 137.14 KB).
NFR-4	Scalability	Multiple users and increasing per-user data supported without correctness issues; read-heavy endpoints cached.	No per-user data leakage under concurrent load; routing layer sustains ≥ 70 RPS with <code>p95</code> ≤ 10 ms (measured: 73.8 RPS, p95 3.78 ms).
NFR-5	Maintainability	Strict separation of concerns; cyclomatic complexity bounded.	Average CC $\leq$ grade B (radon); no single function $>$ grade D; measured: avg A (2.98) , one outlier at D (27).
NFR-6	Extensibility	Adding a new health data source requires no edits to existing modules.	Extension cost = +1 class, 0 edits ; <code>test_open_closed_extension_via_new_provider</code> passes in CI.
NFR-7	Reliability	Graceful degradation under partial failures; health probes exposed.	Partial-failure response (non-5xx) on dashboard when one domain is degraded; <code>/health</code> and <code>/ready</code> endpoints return 200 / 503 correctly.
NFR-8	Testability	Core logic unit-testable; CI enforces a coverage floor.	Backend line coverage $\geq 60\%$ (CI gate <code>-cov-fail-under=60</code>); measured: 76% , 56/56 tests passing in ~ 1 s.
NFR-9	Observability	Every request carries a correlation ID; Prometheus metrics exposed.	100% of requests carry <code>x-request-id</code> by construction; <code>/metrics</code> exposes ≥ 2 metric families (measured: 2 : counter + histogram).

2.3 Key architecturally significant requirements (ASRs)

A requirement is architecturally significant when satisfying it forces a structural decision that cannot be easily refactored later, rules out otherwise attractive alternatives, or has system-wide ripple effects. The six below meet that bar.

1. **Authentication + per-user data isolation** (FR-1/FR-2, NFR-1)

Why it is a driver: Health data is private; a data-isolation failure is a security incident, not a bug. Allowing direct client-to-Firestore writes would scatter auth logic across the frontend and require maintaining Firestore security rules in sync with business rules.

Structural decision forced: All writes go through a backend that verifies the Firebase ID token and uses the extracted `uid` to scope every Firestore path (`users/{uid}/...`). The `get_current_user` FastAPI dependency is wired into every route via `Depends`, making accidental unauthenticated access a compile-time mistake rather than a runtime gap.

2. **Server-side totals and Pydantic validation** (FR-4/5/6, NFR-2)

Why it is a driver: If the backend accepted client-computed nutrition totals, a tampered client could store arbitrary macro values. Pydantic validation at the route boundary is the only place guaranteed to run regardless of client behaviour.

Structural decision forced: Business logic (total computation) must live in the service layer, not in routes or repositories. This drove the four-layer architecture; removing the service layer would require pushing logic into routes (breaking testability) or trusting clients (breaking integrity).

3. **ML inference on the hot request path** (FR-7, NFR-3)

Why it is a driver: ONNX inference is CPU-bound and takes hundreds of milliseconds. Running it synchronously on FastAPI's single-threaded async event loop would block every concurrent request—all other users would see elevated latency during any image upload.

Structural decision forced: Inference is offloaded via `asyncio.to_thread`, introducing an explicit concurrency seam in the meals service. The nutrition lookup chain (Strategy + Chain-of-Responsibility) was extracted from the service because adding new food databases would otherwise require editing inference-adjacent code.

4. **Extensible nutrition lookup** (NFR-6)

Why it is a driver: Nutrition accuracy is a product quality concern that will improve over time (adding regional databases, external APIs). Hard-coding the lookup order inside a service file means each improvement is a risky diff on business-logic code.

Structural decision forced: The `NutritionProvider` protocol and `NutritionResolver` chain (ADR-0004). Any design that keeps the lookup inside a single function fails NFR-6 because the extension cost is always ≥ 1 edit.

5. **Cache-aside for read-heavy aggregation endpoints** (FR-10, NFR-4)

Why it is a driver: The dashboard and analytics services re-read every meal and workout for the user on every request. Under any real load this becomes a Firestore read amplification problem. Without a cache, horizontal scaling only moves the bottleneck to Firestore quotas.

Structural decision forced: An explicit `Cache` protocol with pluggable backends (in-memory for the monolith, Redis-ready for multi-instance). Services depend on the protocol, not a concrete backend—Dependency Inversion Principle applied at the infrastructure boundary.

6. **Progress analytics read model** (FR-12, NFR-3/4)

Why it is a driver: Analytics queries span arbitrary date ranges over unbounded per-user his-

tory. A naive scan-and-aggregate on every request grows linearly with user data volume, violating NFR-3.

Structural decision forced: Firestore predicate push-down (**where** clauses) and write-time denormalization (items embedded in meal documents, ADR-0003) so that list queries never fan out into per-document sub-collection reads. Together these keep the analytics service read cost at $O(\text{range size})$ rather than $O(\text{total history})$.

2.4 Subsystem overview

Decomposition principle. Subsystems are grouped by *data ownership and capability boundary*, not by technical layer. Each domain service owns one Firestore collection namespace and exposes one cohesive set of operations on it; no two services read or write the same collection. The frontend is separated because it runs in an untrusted execution environment (the browser) and must never hold service-account credentials. Image recognition and nutrition lookup are extracted because they carry their own heavyweight dependencies (ONNX runtime, food databases) whose failure modes and extension cost differ from plain CRUD, and they are only consumed by the meals pipeline.

Cross-cutting interfaces. Three concerns span every subsystem and are implemented once rather than replicated:

- **Authentication** — `get_current_user` (auth subsystem) is injected via FastAPI `Depends` into every domain route; no domain service re-implements token verification.
- **Error propagation** — all domain errors are subclasses of `DomainError` (errors subsystem); a single handler in `main.py` maps them to HTTP status codes, so domain services never import from the transport layer.
- **Observability** — correlation-ID middleware, structured logging, and Prometheus metrics are registered once on the FastAPI app; every request is automatically traced and timed regardless of which domain route handles it.

Subsystem	Role	Interfaces with
Web frontend	All user-facing interactions: auth flows, dashboard, logging, analytics, recommendations, plans.	Authentication (token acquisition); all backend API routes (via <code>apiFetch</code>).
Authentication	Firebase ID token issuance (client SDK) and verification (Admin SDK via <code>get_current_user</code>).	All domain services (cross-cutting); Firebase Auth cloud service.
Meals service	CRUD for meal records; delegates image analysis and nutrition lookup.	Image recognition; nutrition lookup; persistence; authentication.
Workouts service	CRUD for workout records.	Persistence; authentication.
Goals service	Manage the single active goal per user.	Persistence; authentication; recommendations (reads active goal).
Dashboard service	Aggregate daily calories-in/out, macro breakdown, counts; cached.	Persistence; caching subsystem; authentication.

Subsystem	Role	Interfaces with
Profile service	Store and retrieve user profile (demographics, dietary preferences).	Persistence; authentication; recommendations (reads profile).
Analytics service	Compute trend metrics over a date range; predicate push-down to Firestore.	Persistence; caching subsystem; authentication.
Recommendations	Rule-based workout plan generation using goal + history; Strategy pattern.	Goals service; profile service; persistence; authentication.
Plans service	Admin-managed global workout plan catalogue; admin auth via custom header.	Persistence; authentication.
Image recognition	ONNX MobileNet food classification; async thread-pool offload.	Meals service (internal call only); ONNX runtime (file dependency).
Nutrition lookup	Tiered food-name-to-macro resolution via Chain-of-Responsibility.	Meals service (internal call only); food database files.
Persistence	Firebase Firestore read/write under <code>users/{uid}/</code> and <code>plans/</code> .	All domain services (cross-cutting); Firebase Firestore cloud service.
Observability	Prometheus metrics, structured JSON logging, correlation-ID propagation.	All subsystems (cross-cutting, registered once on the FastAPI app).

3 Task 2: Architecture Framework

3.1 Stakeholders

Stakeholder	Primary concerns
End user (health tracker)	Fast entry, accurate nutrition totals, trend visibility, private health data, user-friendly UI.
Plan Admin	Clear content management paths, reliable data integrity, and a straightforward way to add/update workout plans.
Development team	Clear module boundaries, local dev workflow, testable modules, straightforward extension paths.

3.2 Architecture Views

3.2.1 Architectural Overview

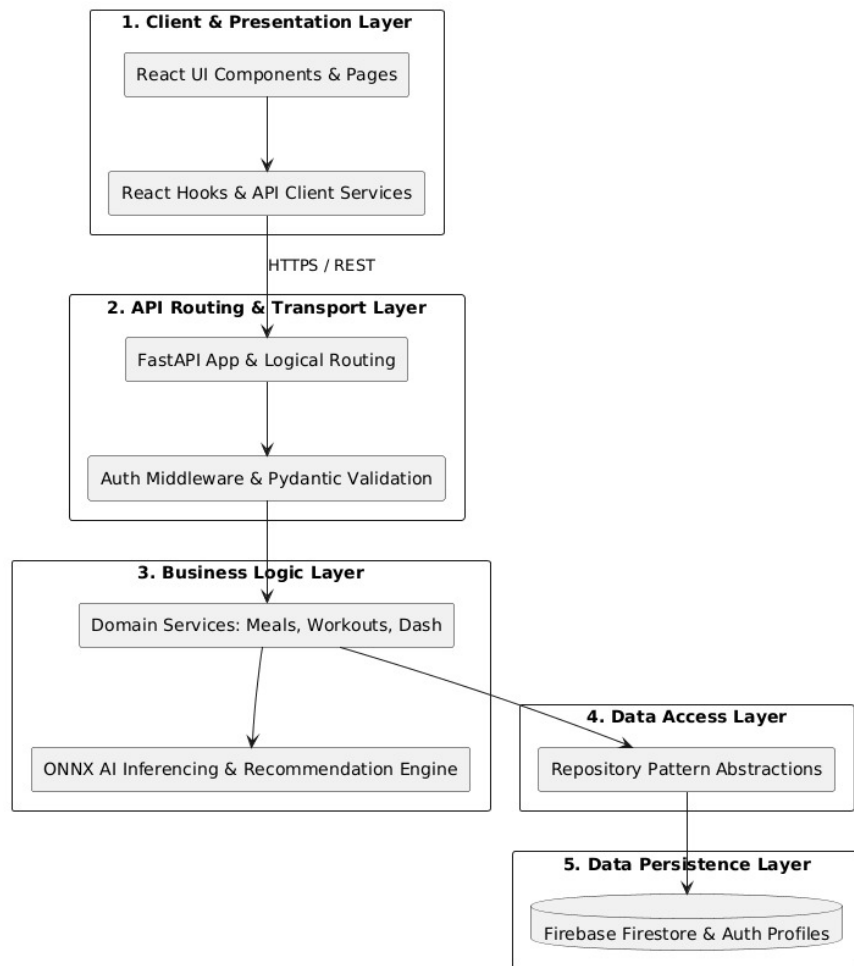


Figure 1: High-level architectural overview of HealthSync.

3.2.2 API Design View

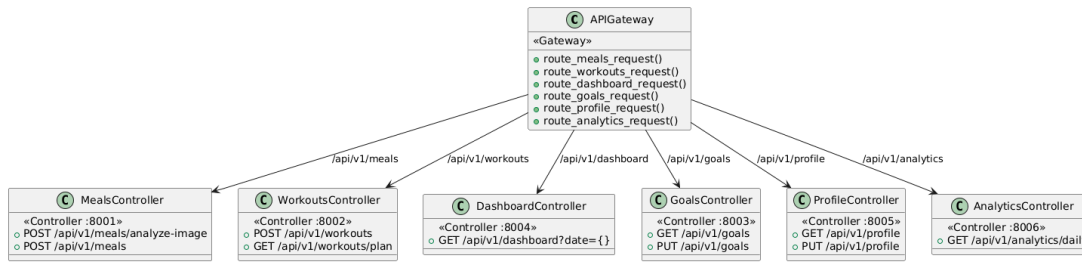


Figure 2: REST API design view

3.2.3 Frontend and Backend Structure

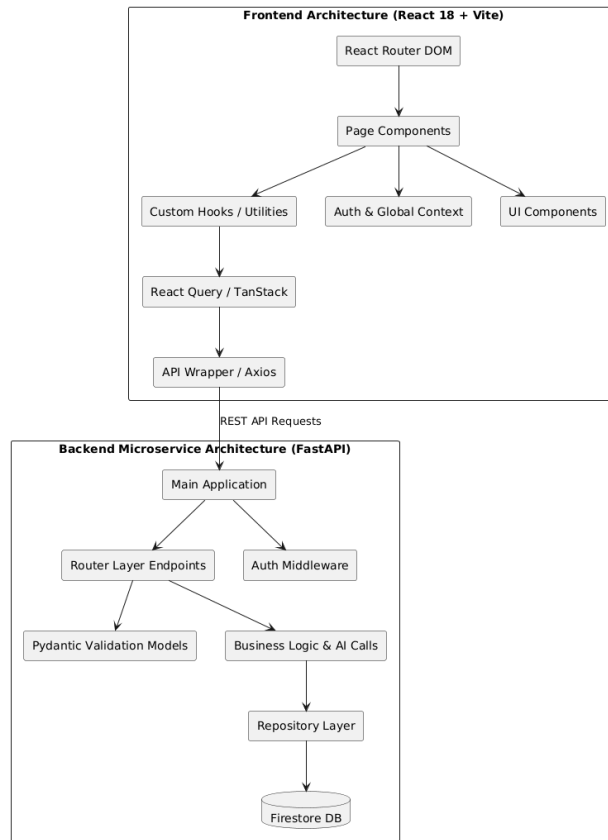


Figure 3: Component/layer diagram

3.2.4 Frontend C3 Diagram

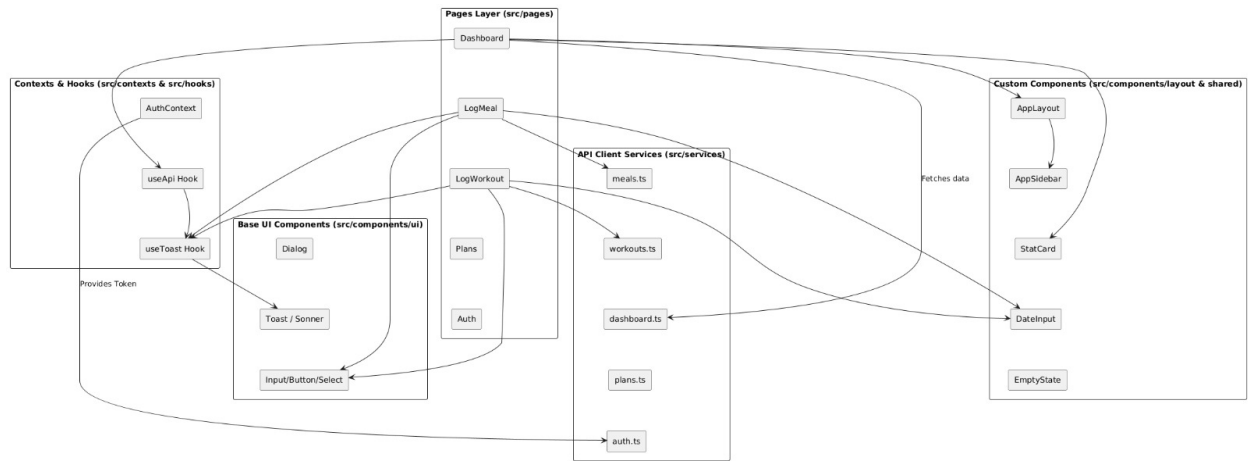


Figure 4: C3 diagram for the frontend

3.2.5 Backend C3 Diagram

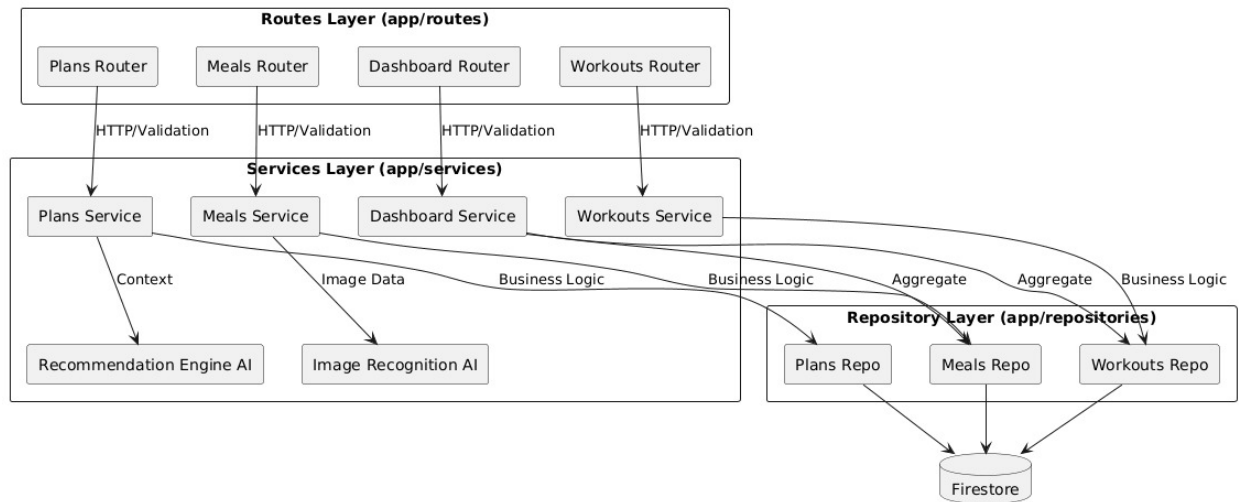


Figure 5: C3 diagram for the backend

3.2.6 Interaction Flow

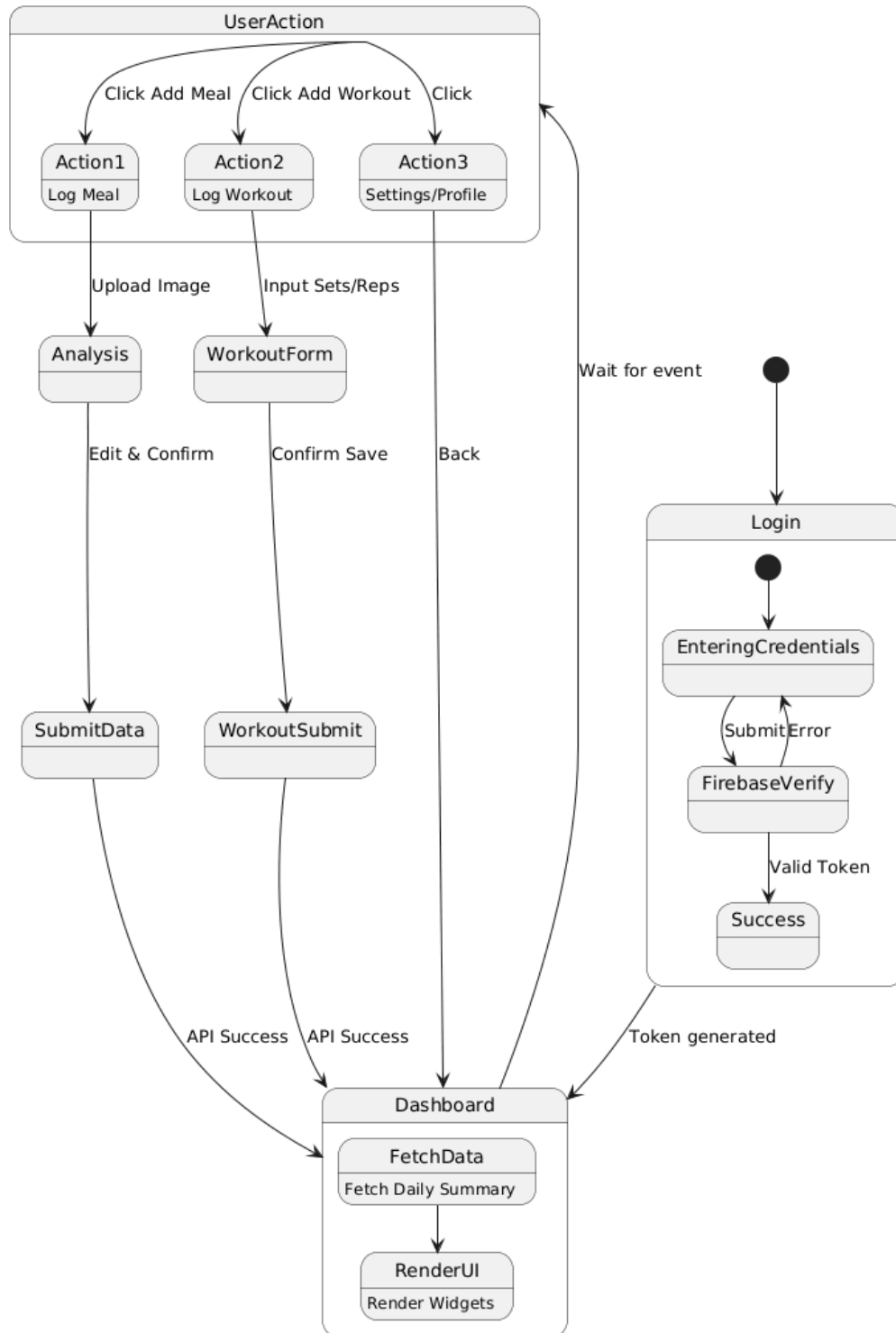


Figure 6: End-to-end interaction flow

3.2.7 Meal Logging Sequence

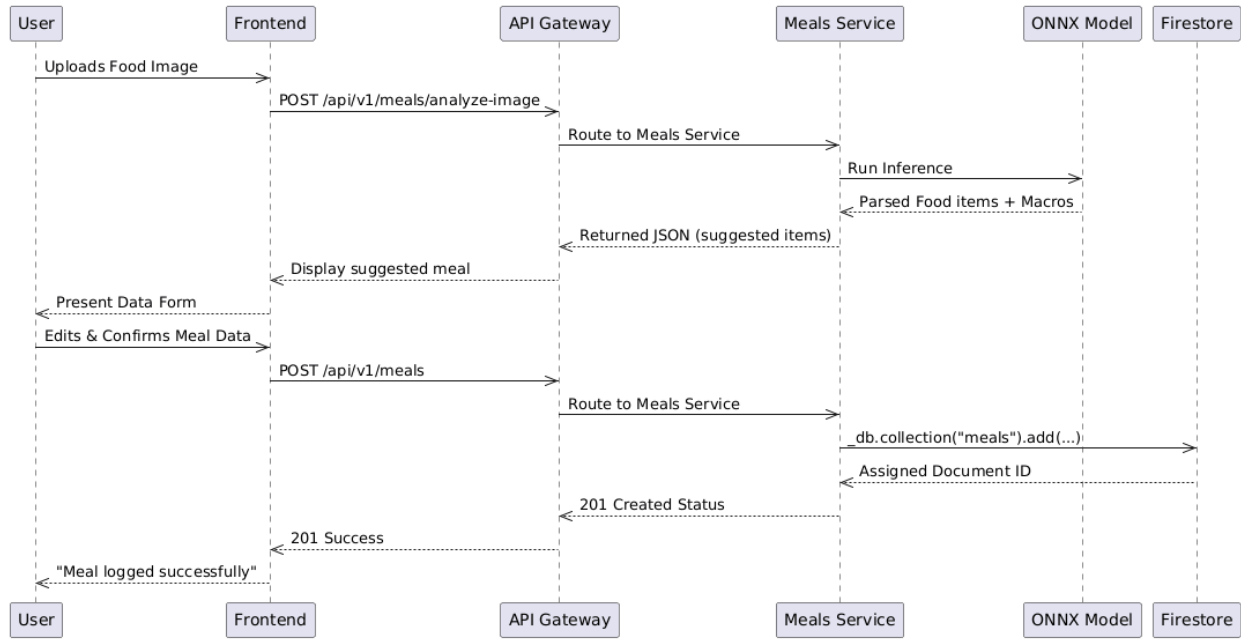


Figure 7: UML sequence diagram for the Log Meal flow

3.2.8 Workout Logging Sequence

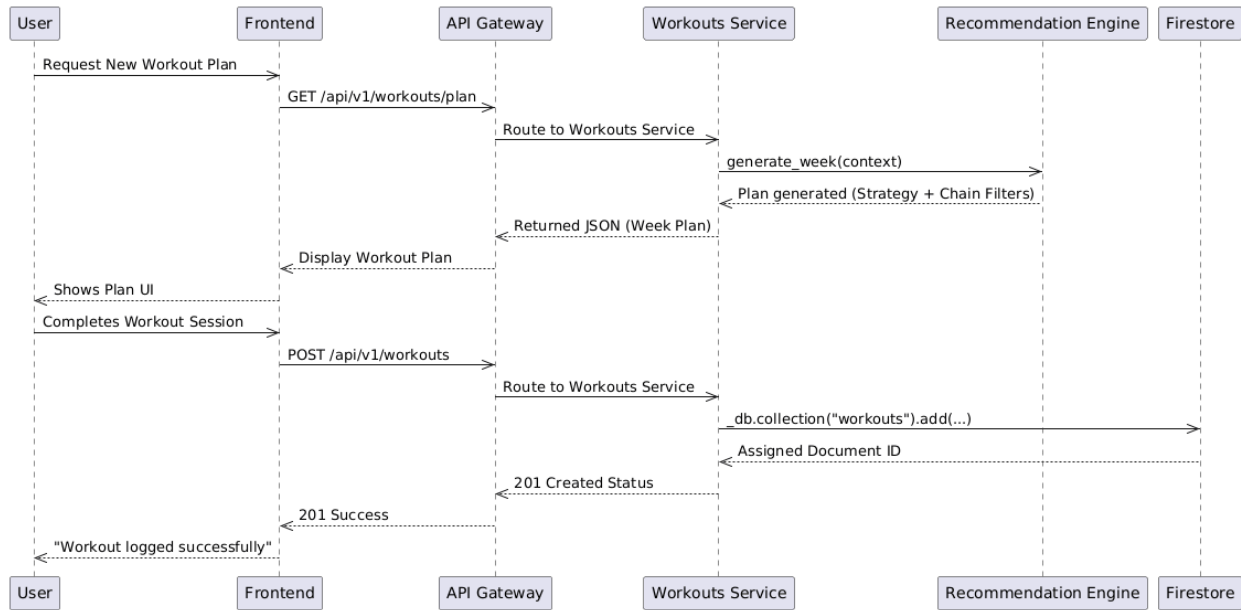


Figure 8: UML sequence diagram for the Log Workout flow

3.3 Major Design Decisions (Architecture Decision Records)

3.3.1 ADR-0001: Strict layered monolith architecture

Status: Accepted (layered monolith)

Context: HealthSync spans multiple feature domains (meals, workouts, goals, dashboard, profile, analytics, recommendations, plans). Early development called for clear module ownership and easy local setup, while the proposal simultaneously required demonstrating a decomposable service boundary. Premature distribution would have added inter-service networking and distributed tracing overhead before the domain models had stabilised.

Decision: Deliver the prototype as a single FastAPI application organised in a strict *routes* → *schemas* → *services* → *repositories* layering. Each layer may only import from the layer directly below it.

Consequences:

- (+) Single process: one port, one log stream, one-command start.
- (+) Layer discipline means each domain can be extracted to its own service with changes confined to one layer.
- (−) A process-level bug (e.g., OOM, unhandled exception) can affect all feature domains. Mitigated by the domain error hierarchy (see ADR-0002) and layer isolation.
- (−) Horizontal scaling requires scaling the entire process; individual aspects (e.g., image analysis) cannot be scaled independently in the monolith.

3.3.2 ADR-0002: Authenticated backend API boundary for all user data writes

Status: Accepted

Context: Meal, workout, goal, and profile records are private health data. The alternative of direct client-side Firestore writes would require maintaining Firestore security rules in sync with client logic and would allow a tampered client to forge nutrition totals or access another user's partition.

Decision: All data mutations pass through FastAPI route handlers. The frontend obtains a short-lived Firebase ID token and `apiFetch` attaches it as a Bearer header. The backend `get_current_user` dependency (in the auth/security subsystem) verifies the token via the Firebase Admin SDK and extracts the verified `uid`, which is the only identity used to scope Firestore reads and writes. The client SDK is used for authentication only; it never touches Firestore directly. diagrams.

Consequences:

- (+) Auth, validation, and logging are enforced at one trusted point; routes cannot accidentally bypass them (FastAPI `Depends` wiring makes omission a type error).
- (+) Server-side nutrition total computation eliminates the risk of a manipulated client payload - a key requirement for data integrity (NFR-2).
- (+) Service layer is testable without a live Firestore instance (`uid` can be a plain string in tests).
- (−) Prototype requires both frontend and backend servers running concurrently.
- (−) One additional network hop compared to direct Firestore client writes.

3.3.3 ADR-0003: Denormalised Firestore meal document

Status: Accepted

Context: The initial Firestore model stored meal items in a sub-collection. Listing a user’s meal history therefore required $N+1$ reads: one collection scan to get N meal summaries, plus one sub-collection read per meal to get items. At page size 100, this was 101 Firestore reads per list call - a direct violation of NFR-3 (sub-linear read cost).

Decision: Embed food items as an `items[]` array directly in the parent meal document at write time, alongside server-computed aggregate totals. Meal summary and item detail are now co-located in a single document. A backward-compatibility read path in the meals service handles any legacy sub-collection rows.

Consequences:

- (+) Listing N meals costs exactly **1 Firestore read** instead of $N+1$, a $100\times$ reduction at $N=100$.
- (+) Removes the async fan-out; the service layer simplifies from a gather step to a single collection scan.
- (–) Meal documents grow with item count; the 1 MB Firestore document limit bounds the maximum items per meal.

3.3.4 ADR-0004: Strategy + Chain-of-Responsibility for nutrition lookup

Status: Accepted

Context: The image analysis endpoint must map a detected food name (e.g., “samosa”, “banana”) to calorie and macro estimates. An initial design hard-coded the priority order and fallback logic inside the meals service. Adding a new food database required editing that file, violating the Open/Closed Principle and making nutrition accuracy improvements a risky diff on a business-logic file.

Decision: Extract the lookup into a `NutritionProvider` protocol with a single `lookup(name)` method, and a `NutritionResolver` that walks a prioritised chain of providers, short-circuiting on the first non-`None` hit. Three providers ship: an Indian-cuisine database (highest priority), a global common-foods database, and a `DefaultEstimateProvider` as a terminal fallback. The chain is wired once at startup in the nutrition subsystem; new sources require only a new class and an insertion point—no edits to existing providers or the resolver.

Consequences:

- (+) Extension cost is just adding a new class.
- (+) Chain short-circuits on the first hit; the expensive `DefaultEstimateProvider` is only reached for unknown foods.
- (–) Chain order is configuration, not type-system enforced; a misconfigured priority order silently returns lower-accuracy data.

3.3.5 ADR-0005: Graceful dashboard degradation under partial service failures

Status: Accepted

Context: The dashboard aggregates meals, workouts, goals, and profile data in a single response. In a multi-service architecture, if any upstream times out, a naive synchronous gather will block the entire dashboard until the slowest dependency responds or times out. Even in the monolith, a slow Firestore read for one domain should not stall the response for the others. Users should see a partial but correct summary rather than a full failure (NFR-7).

Decision: In the layered monolith: the dashboard service queries each domain independently with short timeouts; missing data produces a zero-valued field rather than an exception. The cache-aside layer (in the caching subsystem) absorbs repeated reads: dashboard responses are cached with a TTL so a slow upstream affects only the first uncached request.

Consequences:

- (+) Users can still view available health data even when one domain's backend is degraded.
- (+) Cache-aside (TTL-based) prevents one slow Firestore read from blocking every user request on the hot dashboard endpoint.
- (−) Partial responses require the frontend to handle missing fields gracefully.

4 Task 3: Architectural Tactics and Patterns

4.1 Architectural tactics

Tactic	Quality attr.	How it is applied	Where in code
Authenticate at API boundary	Security	<code>get_current_user</code> verifies the Firebase ID token; any missing or invalid token raises <code>AuthorizationError</code> which maps to HTTP 401 before the route handler body executes.	<code>app/config/security.py</code> ; injected via <code>Depends(get_current_user)</code> in every route file under <code>app/routes/</code> .
Validate at multiple boundaries	Integrity	Frontend client-side validation blocks submission for UX. Backend Pydantic schemas enforce non-negative values, max lengths, regex patterns, and bounded list sizes as the authoritative constraint.	<code>app/schemas/*.py</code> (backend); <code>frontend/src/pages/LogMeal.tsx</code> and sibling pages (frontend form validation).
Server-side aggregate computation	Integrity	Nutrition totals and dashboard aggregates are computed in the service layer on every write; no endpoint accepts pre-computed totals from the client.	<code>app/services/meals_service.py</code> (nutrition totals at write time); <code>app/services/dashboard_service.py</code> (daily aggregates).
Cache-Aside with pluggable backends	Performance / Scalability	Dashboard and analytics responses are stored under a keyed TTL cache; a cache miss falls through to Firestore; invalidation happens at write time.	<code>app/config/cache.py</code> (<code>Cache</code> protocol, <code>InMemoryTTLCache</code> , <code>RedisCache</code>); used in dashboard and analytics services via <code>Depends(get_cache)</code> .
Async thread-pool offload for CPU-bound I/O	Performance	ONNX image inference is dispatched to a thread-pool worker so the FastAPI async event loop is never blocked; other requests are served concurrently during inference.	<code>app/services/meals_service.py</code> : <code>await asyncio.to_thread(recognizer.predict_image_bytes)</code> .

Tactic		Quality attr.	How it is applied	Where in code
Correlation-ID structured logging	+	Observability	HTTP middleware mints a UUID4 <code>x-request-id</code> (or accepts an upstream one), attaches it to every log line, and echoes it on the response header. In microservices mode the gateway propagates the same ID to upstreams.	<code>app/main.py</code> (middleware, lines 55–90); <code>microservices/_base.py</code> + <code>microservices/gateway/proxy.py</code> (comparison branch).
Prometheus process metrics	in-	Observability	A zero-dependency metrics registry counts requests and records latency histograms (11 buckets, 5 ms–10 s) per route; <code>/metrics</code> exposes Prometheus text format without requiring an external agent.	<code>app/observability/metrics.py</code> ; registered via <code>register_metrics_middleware(app)</code> and <code>register_metrics_endpoint(app)</code> at the bottom of <code>app/main.py</code> .
React Query stale-while-revalidate + mutation invalidation	+	Consistency	After every successful mutation (meal log, workout log, goal update) the frontend invalidates the affected query keys, triggering a background refetch so the dashboard and history views reflect the new server state immediately.	<code>frontend/src/hooks/useApi.ts</code> ; mutation callbacks call <code>queryClient.invalidateQueries({query: [...]}))</code> targeting dashboard and history keys.

4.2 Implementation patterns

4.2.1 Pattern 1: Strategy Pattern (Recommendation Engine)

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable. The pattern lets the algorithm vary independently from clients that use it.

Role in HealthSync: The `RecommendationEngine` selects and executes a workout-plan-building algorithm based on the user’s goals and profile, without the engine itself knowing which concrete strategy is active.

Structure:

- **Context** — `RecommendationEngine` (`backend/app/services/recommendation_engine.py`): holds a reference to a `RecommendationStrategy` and delegates generation to it. The private `resolve_strategy(context)` method selects the concrete strategy based on the user’s stated goal.
- **Strategy interface** — `RecommendationStrategy`: declares `build_plan(context)` and `build_week_plan(context)`.

- **Concrete strategies:**

- **StrengthTrainingStrategy** — generates strength-focused plans
- **CardioEnduranceStrategy** — generates cardio/endurance plans
- **WeightLossStrategy** — generates calorie-deficit-oriented plans

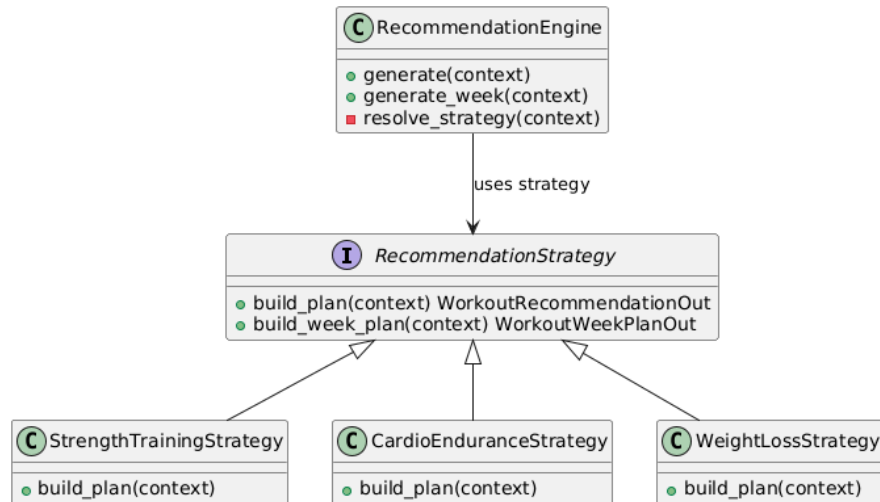


Figure 9: Strategy Pattern applied to the Recommendation Engine.

How it addresses NFRs:

- **Extensibility:** adding a new goal type (e.g., *Flexibility*) requires only a new class implementing RecommendationStrategy.
- **Maintainability:** each strategy is a self-contained module with a single responsibility; they can be read, tested, and evolved independently.
- **Testability:** the strategy interface can be mocked in unit tests RecommendationEngine without instantiating any real plan-building logic.

4.2.2 Pattern 2: Repository Pattern

Role in HealthSync: All Firestore interactions are channelled through repository classes. Service code never constructs Firestore document paths or executes queries directly, making the data access layer independently testable and swappable.

Structure:

- **Abstract base** — BaseRepository: declares the common interface `get_by_id(id)`, `add(document)`, and `update(id, document)`.
- **Concrete repositories:**
 - **MealsRepository** — adds `get_meals_by_date(user_id, date)`; encapsulates the Firestore path and the date-filter predicate.

- `WorkoutsRepository` — adds `get_workouts_in_range(user_id, start_date, end_date)`; encapsulates the Firestore path and range query.
- Analogous repositories exist for Goals, Profile, Plans, and Recommendations.

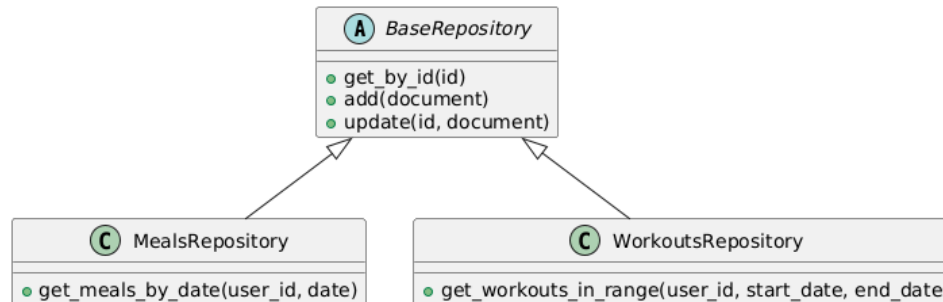


Figure 10: Repository Pattern: `BaseRepository` defines the common data-access interface; `MealsRepository` and `WorkoutsRepository` extend it with domain-specific query methods, hiding all Firestore path logic from the service layer.

How it addresses NFRs:

- **Testability:** Easy to mock the actual db implementation; service tests never touch real Firestore.
- **Maintainability:** Firestore path construction is in one place; renaming a collection requires editing only the relevant repository file.
- **Extensibility:** adding a new collection (e.g., weight logs) means creating a new repository subclass without touching any existing service.

Other design patterns present

- **Chain of Responsibility** (`backend/app/services/nutrition/providers.py`)
`NutritionResolver` walks a list of `NutritionProvider` objects in priority order and short-circuits on the first non-None hit (Indian foods → global foods → default estimate). Each provider decides independently whether it can handle the request; none has knowledge of its neighbours.
- **Facade** (`backend/app/services/*.py`)
Each service module (e.g., `meals_service.py`, `dashboard_service.py`) acts as a Facade: it presents a simple, high-level interface to the route layer (`log_meal()`, `get_dashboard()`) while internally coordinating the repository, cache, image recogniser, and domain error hierarchy. The route handler never needs to know about Firestore or ONNX.
- **Observer** (`frontend/src/` — TanStack React Query)
After every successful mutation (meal log, workout log, goal update), the frontend ensures React Query re-fetches all registered observers of the invalidated key, ensuring the dashboard and history views automatically reflect the latest server state without manual re-rendering.

5 Task 4: Prototype Implementation and Architecture Analysis

5.1 Implemented prototype

Today's Overview on the Dashboard

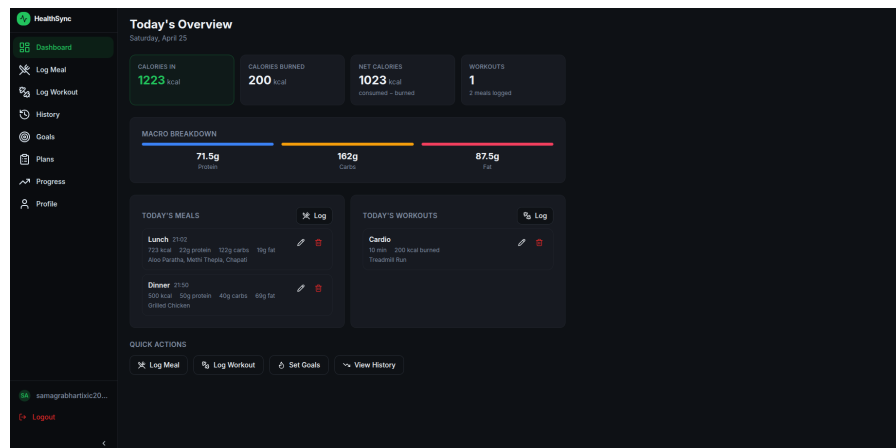


Figure 11: Dashboard UI with Today's Overview.

Log Meal form with image analysis

The screenshot shows the 'Log Meal' form in the HealthSync app. The form is titled 'Log Meal' and includes a subtitle 'Add one or more foods and HealthSync will log the meal for your dashboard'. It is divided into several sections: 'Meal Details' with fields for Type (Lunch), Date (04/25/2020), and Time (09:51 PM); 'Image Analysis (Optional)' with a 'Meal Image' field and a 'Choose File' button; 'Food Items' with a table for logging food items (Name, Calories, Protein, Carbs, Fat) and an 'Add Item' button; and a summary section at the bottom showing 'CALORIES 326 kcal', 'PROTEIN 7.5g', 'CARBS 47g', and 'FAT 12g'. The form also includes 'Save Meal' and 'Cancel' buttons at the bottom.

Figure 12: Log Meal form with image analysis results.

Profile page with editable fields

The screenshot shows the 'Profile' page of the HealthSync application. The left sidebar contains navigation links: Dashboard, Log Meal, Log Workout, History, Goals, Plans, Progress, and Profile (which is highlighted). The main content area is titled 'Profile' and contains a 'Personal Details' section. This section includes input fields for 'Full Name' (with placeholder 'e.g. Alex Kumar'), 'Age' (with placeholder 'e.g. 22'), 'Gender' (a dropdown menu with 'Select'), 'Height (cm)' (with placeholder 'e.g. 175'), and 'Weight (kg)' (with placeholder 'e.g. 72.5'). Below these are text areas for 'Dietary Preferences (comma-separated)' (with placeholder 'Food') and 'Allergies (comma-separated)' (with placeholder 'food'). A green 'Save Profile' button is at the bottom of the form. The bottom of the sidebar shows the user's name 'samagrabhartix20...' and a 'Logout' button.

Figure 13: Profile page with editable fields.

History page

The screenshot shows the 'History' page of the HealthSync application. The left sidebar is the same as in Figure 13, with 'History' highlighted. The main content area is titled 'History' and has two tabs: 'Meals' and 'Workouts' (which is selected). Below the tabs, there is a list of three workout entries, each with a title, date, duration, and calories burned, and a delete icon.

Activity	Date	Duration	Calories Burned	Action
Cardio	2020-04-20	10 min	200 kcal burned	Delete
Strength	2020-04-21	5 min	100 kcal burned	Delete
Cycling	2020-04-22	40 min	1000 kcal burned	Delete

Figure 14: History page showing past workouts.

Workout recommendation page

HealthSync

Dashboard

Log Meal

Log Workout

History

Goals

Plans

Progress

Profile

Plans

Choose your training goals and HealthSync will match pre-planned workouts you can start immediately.

Health Goals

Strategy

Strength

Cardio Endurance

Weight Loss

Flexibility

Experience Level

Beginner

Time Budget (minutes)

30

Workout Days per Week

3

Available Equipment

dumbbells, treadmill, mat

Injuries or Constraints

knee, shoulder

Preferred Workout Types

strength training, running, yoga

Profile

Age: not set | Gender: not set

Matched Plans

10 plans fit your current goals.

Weight Training for Senior Citizens

Best match

Lower-impact strength work for stability, controlled range, and confidence.

Strength 30 min 2 days/week

Ages 60-100

1. Chair Squat
Strength | 3 x 8

2. Supported Dumbbell Row
Strength | 3 x 10

3. Seated Shoulder Press
Strength | 2 x 8

4. Standing Calf and Hip Stretch
Yoga | 8 min

Start this Plan

Calisthenics for beginners

Best match

Strength 30 min 3 days/week

Ages 18-25 Trainer plan

1. Incline Push Up
Strength | 2 x 2

Start this Plan

4 Day Beginner Calisthenics

Best match

A bodyweight strength plan for building control, pushing strength, pulling strength, and core stability.

Strength 35 min 4 days/week

Ages 14-60

1. Incline Push-Up
Strength | 3 x 10

2. Assisted Bodyweight Row
Strength | 3 x 8

3. Bodyweight Squat
Strength | 3 x 12

4. Dead Bug
Strength | 3 x 10

Start this Plan

Calisthenics for intermediates

Best match

Strength 30 min 4 days/week

Ages 18-35 Trainer plan

1. Incline Push-up
Strength | 3 x 9

Start this Plan

Weight Training for Adults

Good match

A compound lifting session built around push, pull, squat, and hinge patterns.

Strength 45 min 4 days/week

Ages 18-59

1. Bench Press
Strength | 4 x 8

2. Goblet Squat
Strength | 3 x 10

3. Bent-Over Row
Strength | 3 x 10

4. Romanian Deadlift
Strength | 3 x 10

Start this Plan

6 Day Upper Lower Strength Split

Good match

A higher-frequency strength split for lifters who want structured volume across upper and lower body days.

Strength 55 min 6 days/week

Ages 18-55

1. Barbell Squat
Strength | 4 x 6

2. Bench Press
Strength | 4 x 6

3. Dumbbell Row
Strength | 4 x 8

4. Romanian Deadlift
Strength | 3 x 8

Start this Plan

samagrabhartvic2...

Logout

Figure 15: Workout recommendation page showing available plans.

Log Workout form

The screenshot shows the 'Log Workout' form in the HealthSync app. The left sidebar contains navigation options: Dashboard, Log Meal, Log Workout (selected), History, Goals, Plans, Progress, and Profile. The main content area is titled 'Log Workout' and includes a sub-header 'Add one or more exercises and HealthSync will total the workout for your dashboard.' Below this is a form for logging a workout. It includes a 'Date' field set to '04/25/2026'. Under 'Exercise 1', there is a 'Type' dropdown set to 'Strength' and an 'Exercise Name' field set to 'Push Ups'. Below these are fields for 'Sets' (3), 'Reps' (15), and 'Weight' (65 kg). There are also fields for 'Calories (kcal)' (0) and 'Notes (optional)' (optional). At the bottom, there are two summary fields: 'TOTAL DURATION' (0 min) and 'TOTAL BURNED' (0 kcal). At the very bottom of the form are 'Save Workout' and 'Cancel' buttons. The bottom of the screen shows a user profile section with the name 'samagrabharti20...' and a 'Logout' button.

Figure 16: Log Workout form.

Progress view with charts

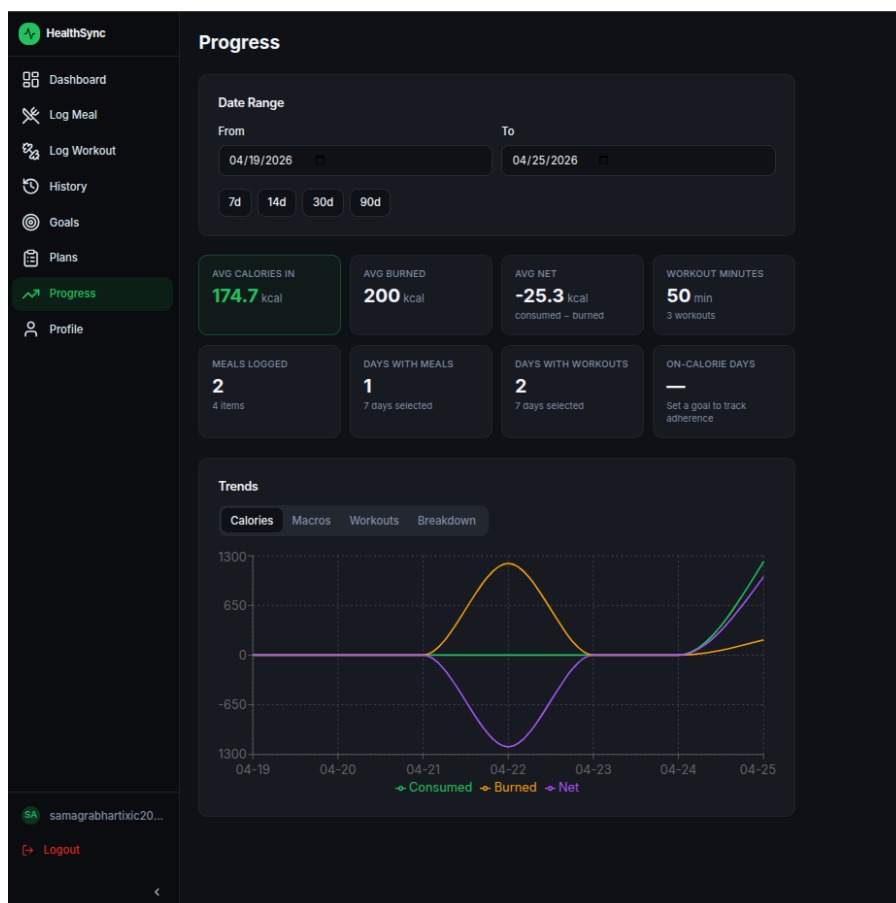


Figure 17: Progress view with charts.

5.2 Architecture comparison: layered monolith vs. microservices

The implemented architecture (`layered` branch) is a **strict layered monolith**. The comparison architecture (`comparison` branch) decomposes the same feature set into **six independently-runnable FastAPI services** behind a gateway.

Concern	Layered monolith (implemented)	Microservices (comparison branch)
Security	Single token-verification dependency; one CORS configuration; one TLS endpoint.	Each service independently verifies tokens (same Firebase Admin SDK); gateway owns CORS; larger attack surface but more granular access control.
Data integrity	Single process; service-to-service calls are in-process function calls with no serialisation overhead.	Service-to-service calls cross network boundaries; JSON serialisation adds latency but makes contracts explicit.
Maintainability	Strict layer separation is enforced by module imports; changes to one feature area can still accidentally reach another via shared utilities.	Physical process isolation prevents cross-domain import accidents; each service has its own <code>requirements.txt</code> and deployment lifecycle.
Performance	No inter-service network hops. Measured: mean 2.84 ms , p95 4.08 ms on <code>GET /api/v1/meals</code> (80 RPS, 15 s bench). Cache-aside is per-process (in-memory).	One gateway $\rightarrow$ upstream HTTP hop adds +1.27 ms mean (+45%) , p95 5.32 ms on the same route. Cache must be shared (Redis) across service instances.
Scalability	73.8 RPS (p95 3.78 ms) at the routing layer (<code>/health</code> , 0 errors). Scaling requires scaling the entire process; a single hot path (e.g., image analysis) cannot scale independently.	73.9 RPS (p95 2.73 ms) at the gateway on the same route. Individual services can be scaled horizontally per load profile (e.g., scale the analytics or image-analysis service independently).
Operational complexity	One process, one port, one log stream.	Seven processes, seven ports, distributed tracing required for end-to-end request tracing.
Delivery speed	Single-command start: <code>uvicorn app.main:app</code> .	Seven-command start abstracted by <code>run_microservices.py</code> or Docker Compose.

Trade-off summary: The layered monolith is strictly superior for development velocity, debugging simplicity, and in-process performance. The microservices decomposition becomes valuable when individual services have genuinely different scaling needs or when teams grow large enough that independent deployment cycles reduce coordination overhead. For the current team size and stage, the monolith is the correct default; the microservices branch serves as a forward migration target.

5.3 NFR quantification

NFR	Metric	Result	Method
Performance	Firestore reads per GET /api/v1/meals at pageSize=100	101 → 1 reads	Items are denormalized into the meal document at write time (meals service). At $N=100$: $101 \rightarrow 1$ ($100\times$ fewer reads).
Performance (frontend bundle)	Initial JS bundle size (gzip)	137.14 KB	Route-level code splitting via <code>React.lazy()</code> and <code><Suspense></code> reduces the initial parse budget.
Maintainability	Average cyclo-matic complexity (CC) across backend/app/	A (2.98) average; single D (27) outlier	<code>radon cc backend/app -a -total-average</code> ; Sole outlier: <code>_compute_daily_progress</code> in the analytics service (rank D, CC=27). Total SLOC: 2,393 across 52 files; largest file <code>recommendation_engine.py</code> at 500 SLOC.
Testability	Backend line coverage; test count; suite runtime	76% coverage, 56/56 passing, ~1.0 s	<code>pytest -cov=app -cov-report=term-missing</code> : 56 tests across 16 files, 1,809 statements covered, 430 missed. Suite runs in ~1.0 s (mocked Firestore, no real I/O). CI enforces <code>-cov-fail-under=60</code> .
Extensibility	Cost to add a new nutrition data source (OCP compliance)	+1 class, 0 edits	<code>test_open_closed_extension_via_new_provider</code> in the backend test suite plugs in a <code>SushiProvider</code> class without editing <code>IndianNutritionProvider</code> , <code>GlobalCommonFoodsProvider</code> , <code>DefaultEstimateProvider</code> , or <code>NutritionResolver</code> . Test is green in CI; any future change that closes the chain will fail CI automatically.
Observability	Prometheus metric families; log fields per request; request-ID coverage	2 families; 6 fields; 100%	<code>/metrics</code> exposes: <code>http_requests_total</code> (counter) and <code>http_request_duration_seconds</code> (histogram, 11 buckets: 5 ms–10 s). Every log line carries: <code>request_id</code> , <code>method</code> , <code>path</code> , <code>status_code</code> , <code>duration_ms</code> , <code>client_ip</code> . Every request mints or inherits an <code>x-request-id</code> —100% coverage by construction. In the <code>comparison</code> branch the gateway propagates the same ID to the upstream service, enabling cross-hop log joins.

NFR	Metric	Result	Method
Scalability (routing layer)	Sustained RPS and p95 latency at the routing boundary (/health, no auth), monolith vs. gateway	Monolith: 73.8 RPS , p95 3.78 ms ; Gateway: 73.9 RPS , p95 2.73 ms (both at 0 errors)	<code>bench.py -rps 80 -duration 15</code> on both stacks; Both saturate the 80 RPS scheduler with zero errors. The gateway is marginally faster on /health because it carries less middleware (no rate-limiter, no domain-error handler). Qualitative scaling advantage of microservices—independent horizontal scaling per service—cannot be captured on a local single-machine bench.

6 Reflection and Lessons Learned

- Layered architecture (routes / schemas / services / repositories) made testing fast and clean
- ADRs were useful for making and communicating decisions, and for future reference
- Nutrition accuracy remains unresolved